

AWS CLI Syntax, Launching EC2 Instances, and Environment-Based Scripting

Course: Cloud Computing **Estimated Time:** 90-120 minutes **Difficulty:** Beginner → Intermediate
Prerequisites: Labs 01-03 completed (AWS account + IAM user + MFA + access key; Docker container; AWS CLI + GitHub configured inside the container).

1. Learning Objectives

By the end of this lab, you will be able to:

1. Read and construct any AWS CLI command by understanding its general syntax structure.
 2. Launch a simple EC2 instance using the CLI (Example 1).
 3. Launch a more configured EC2 instance with tags, security group, and user-data (Example 2).
 4. Store reusable values (like AMI IDs, key names, region) in an **environment file** (`.env`) instead of hardcoding them into scripts.
 5. Explain why `.env` files must be excluded from version control using `.gitignore`.
 6. Write and run a **create script** that launches 2 EC2 instances and a **delete script** that terminates them cleanly.
-

2. Why This Matters

Typing long AWS CLI commands by hand is error-prone and doesn't scale. Real-world cloud engineers write **scripts** that take configuration values (region, instance type, AMI ID) from a **central variables file**, so the same script can be reused across projects, environments, or team members without editing the script itself. Keeping secrets and environment-specific values out of your script — and out of Git — is a core DevOps and security practice you'll use for the rest of your career.

⚠ **Cost Warning:** EC2 instances cost money once running (even Free Tier has limits on hours/month). Always confirm you used a Free Tier-eligible instance type (`t3.micro` or `t3.micro` , depending on your region) and **always run the delete script at the end of this lab**. Leaving instances running is the #1 cause of unexpected AWS bills in student labs.

3. Part A — Understanding AWS CLI Command Syntax

Every AWS CLI command follows the same general structure:

```
aws <service> <operation> [--parameter1 value1] [--parameter2 value2] ... [--region <region>] [--output <format>]
```

Component	Description	Example
<code>aws</code>	The CLI executable itself	<code>aws</code>
<code><service></code>	Which AWS service you're targeting	<code>ec2</code> , <code>s3</code> , <code>iam</code> , <code>sts</code>
<code><operation></code>	The specific action to perform on that service	<code>run-instances</code> , <code>describe-instances</code> , <code>terminate-instances</code>
<code>--parameter value</code>	Named flags that configure the operation (order doesn't matter)	<code>--instance-type t3.micro</code>
<code>--region</code>	(Optional) Overrides your default region from <code>aws configure</code>	<code>--region us-east-2</code>
<code>--output</code>	(Optional) Overrides default output format	<code>--output table</code>

Example: Breaking Down a Real Command

```
aws ec2 describe-instances --region us-east-2 --output table
```

Part	Meaning
<code>ec2</code>	Service = Elastic Compute Cloud
<code>describe-instances</code>	Operation = list/describe existing instances
<code>--region us-east-2</code>	Parameter = look in the N. Virginia region
<code>--output table</code>	Parameter = format results as a readable table instead of raw JSON

Getting Help Without Memorizing Everything

You never need to memorize every parameter. Use built-in help:

```
aws ec2 help           # Lists all EC2 operations
aws ec2 run-instances help # Shows every parameter for this specific operation
```

Press `q` to exit the help viewer.

Useful Global Flags

Flag	Purpose
<code>--dry-run</code>	Tests whether you <i>would</i> have permission, without actually performing the action
<code>--output json \ table \ text</code>	Controls how results are displayed
<code>--query "<JMESPath>"</code>	Filters/extracts specific fields from a JSON response
<code>--profile <name></code>	Uses a named credential profile instead of the default

✓ **Checkpoint:** Run `aws ec2 describe-instances --region us-east-2` and confirm it returns (likely an empty list, since you haven't launched anything yet — that's expected).

4. Part B — Launching Your First EC2 Instance (Example 1: Simple)

We'll start with the absolute minimum needed to launch an instance.

B1. Find a Free-Tier-Eligible AMI (Amazon Machine Image)

AMI IDs are region-specific and change over time, so look up the current one rather than hardcoding an old value. The AWS CLI supports a `free-tier-eligible` filter on `describe-images`, so you can ask AWS directly for AMIs that qualify for the Free Tier instead of guessing:

```
aws ec2 describe-images \
  --owners amazon \
  --filters "Name=name,Values=al2023-ami-*-x86_64" "Name=state,Values=available" "Name=free-tier-eligible,Values=true" \
  --query "sort_by(Images, &CreationDate)[-1].[ImageId,Name]" \
  --output table \
  --region us-east-2
```

This finds the **latest Amazon Linux 2023** AMI that is Free Tier-eligible in `us-east-2`. Copy the `ImageId` (starts with `ami-`) shown in the output.

💡 **Note:** Being on a Free Tier-eligible AMI covers the *software/OS image* itself. You must still pair it with a Free Tier-eligible **instance type** (`t3.micro` or `t2.micro`, depending on your account age/region — see the Cost Warning above) to stay within Free Tier limits. If the filtered query above returns no results in your region, drop the `free-tier-eligible` filter and simply confirm you're launching with a `t3.micro` / `t2.micro` instance type, since nearly all standard Amazon Linux AMIs work with Free Tier instance types.

B2. Create a Key Pair (for SSH access, one-time setup)

```
aws ec2 create-key-pair \
  --key-name ITM0-444-544-ITM0-444-544-lab-keypair \
  --query "KeyMaterial" \
  --output text \
  --region us-east-2 > ITM0-444-544-lab-keypair.pem

chmod 400 ITM0-444-544-lab-keypair.pem
```

🔑 `ITM0-444-544-lab-keypair.pem` is a private key — treat it like a password. Never commit it to Git (we'll enforce this in Part D).

B3. Launch the Simple Instance

```
aws ec2 run-instances \
  --image-id <ami-id-from-B1> \
  --instance-type t3.micro \
  --key-name ITM0-444-544-lab-keypair \
  --count 1 \
  --region us-east-2
```

The output includes an `InstanceId` (e.g., `i-0abcd1234efgh5678`) — copy it.

B4. Confirm It's Running

```
aws ec2 describe-instances \
  --instance-ids <your-instance-id> \
  --query "Reservations[0].Instances[0].[State.Name,PublicIpAddress]" \
  --output table \
  --region us-east-2
```

Wait until `State.Name` shows `running` (it may briefly show `pending` first).

B5. Terminate It (Don't Skip — Avoid Charges)

```
aws ec2 terminate-instances --instance-ids <your-instance-id> --region us-east-2
```

✓ **Checkpoint:** `describe-instances` shows `State.Name` as `terminated` or `shutting-down`.

5. Part C — A More Realistic EC2 Launch (Example 2: Increased Complexity)

Real deployments add **tags** (for identification/billing), a **security group** (firewall rules), and **user-data** (a startup script that runs automatically on first boot).

C1. Create a Security Group

```
aws ec2 create-security-group \
  --group-name ITM0-444-544-lab-sg \
  --description "Allow SSH and HTTP for lab" \
  --region us-east-2
```

Copy the returned `GroupId` (starts with `sg-`).

C2. Add Inbound Rules (SSH + HTTP)

```
aws ec2 authorize-security-group-ingress \
  --group-id <sg-id> \
  --protocol tcp --port 22 --cidr 0.0.0.0/0 \
  --region us-east-2

aws ec2 authorize-security-group-ingress \
  --group-id <sg-id> \
  --protocol tcp --port 80 --cidr 0.0.0.0/0 \
  --region us-east-2
```

⚠ Opening port 22 (SSH) to `0.0.0.0/0` (the whole internet) is acceptable for a short-lived lab instance, but is **not** a production best practice. In a real deployment, restrict this to your own IP address.

C3. Write a User-Data Script

This script runs automatically when the instance boots, installing and starting a simple web server.

```
cat > user-data.sh << 'EOF'
#!/bin/bash
dnf update -y
dnf install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>Hello from $(hostname -f)</h1>" > /var/www/html/index.html
EOF
```

C4. Launch the Configured Instance

```
aws ec2 run-instances \
  --image-id <ami-id-from-B1> \
  --instance-type t3.micro \
  --key-name ITM0-444-544-lab-keypair \
  --security-group-ids <sg-id> \
  --user-data file://user-data.sh \
  --count 1 \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=lab-web-server},
{Key=Course,Value=CloudComputing}]' \
  --region us-east-2
```

What's new compared to Example 1:

Addition	Purpose
--security-group-ids	Attaches firewall rules (SSH + HTTP access)
--user-data file://...	Runs a bootstrap script automatically on first launch
--tag-specifications	Adds searchable metadata (Name, Course) — useful for billing and organization

C5. Verify and Test

Wait ~1-2 minutes for the user-data script to finish, then get the public IP:

```
aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=lab-web-server" \
  --query "Reservations[0].Instances[0].PublicIpAddress" \
  --output text \
  --region us-east-2
```

Open that IP address in a browser — you should see **“Hello from ip-xxx-xxx-xxx-xxx...”**

C6. Clean Up

```
aws ec2 terminate-instances --instance-ids <instance-id> --region us-east-2
```

✓ **Checkpoint:** You saw the “Hello” web page and successfully terminated the instance.

6. Part D — Using an Environment File Instead of Hardcoding Values

Notice how Parts B and C repeated the same AMI ID, key name, region, and security group across commands. Let's centralize these into a `.env` file.

D1. Create the `.env` File

Inside your project/repo folder in the container:

```
nano .env
```

Paste in (replace with your actual values from Parts B and C):

```
AWS_REGION=us-east-2
AMI_ID=ami-xxxxxxxxxxxxxxxx
KEY_NAME=ITM0-444-544-lab-keypair
SECURITY_GROUP_ID=sg-xxxxxxxxxxxxxxxx
INSTANCE_TYPE=t3.micro
INSTANCE_COUNT=1
NAME_TAG=cloud-lab-instance
```

Save and exit.

D2. Load the `.env` File Into Your Shell

Scripts can read this file using a single line:

```
export $(grep -v '^#' .env | xargs)
```

This exports every non-comment line as an environment variable, so `$AWS_REGION`, `$AMI_ID`, etc. become available to any script you run afterward.

Verify it worked:

```
echo $AWS_REGION  
echo $AMI_ID
```

D3. Protect the `.env` File with `.gitignore`

Why this matters: Your `.env` file may contain AMI IDs, security group IDs, and — in more advanced setups — actual secrets. Even values that aren't secret (like an AMI ID) can reveal your account's internal configuration and should stay out of a public repository as good hygiene. If this were a real project handling API keys or database passwords, committing a `.env` file could expose credentials to anyone who views your repo, including automated bots that scan GitHub for leaked secrets within minutes of a push.

Create a `.gitignore` file in the same folder:

```
nano .gitignore
```

Add this line:

```
.env
```

Save and exit. Confirm Git will now ignore it:

```
git status
```

`.env` should **not** appear in the list of files to be committed. You can also verify explicitly:

```
git check-ignore -v .env
```

This should print the `.gitignore` rule that matches it.

✓ **Checkpoint:** `git status` never shows `.env` as a trackable/staged file, even after you `git add .`

💡 **Best Practice:** Commit a `.env.example` file instead — a template with the same variable names but placeholder values (e.g., `AMI_ID=ami-xxxxxxxx`). This lets classmates or graders know what variables are needed without exposing your actual values.

Create it now:

```
cat > .env.example << 'EOF'
AWS_REGION=us-east-2
AMI_ID=ami-xxxxxxxxxxxxxxxx
KEY_NAME=your-keypair-name
SECURITY_GROUP_ID=sg-xxxxxxxxxxxxxxxx
INSTANCE_TYPE=t3.micro
INSTANCE_COUNT=1
NAME_TAG=cloud-lab-instance
EOF

git add .env.example .gitignore
git commit -m "Add environment template and gitignore"
git push
```

7. Part E — Create and Delete Scripts Using the Environment File

Now we'll write two scripts that use `.env` values to launch **2 EC2 instances** and cleanly terminate them.

E1. `create_instances.sh`

```
nano create_instances.sh
```

Paste in:

```
#!/bin/bash
set -e # Exit immediately if any command fails

# Load environment variables
export $(grep -v '^#' .env | xargs)

echo "Launching $INSTANCE_COUNT instance(s) in $AWS_REGION..."

OUTPUT=$(aws ec2 run-instances \
  --image-id "$AMI_ID" \
  --instance-type "$INSTANCE_TYPE" \
  --key-name "$KEY_NAME" \
  --security-group-ids "$SECURITY_GROUP_ID" \
  --count "$INSTANCE_COUNT" \
  --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=$NAME_TAG}]" \
  --region "$AWS_REGION" \
  --query "Instances[*].InstanceId" \
  --output text)

echo "Launched instance IDs:"
echo "$OUTPUT"

# Save instance IDs to a file so the delete script can find them later
echo "$OUTPUT" | tr '\t' '\n' > instance_ids.txt
echo "Instance IDs saved to instance_ids.txt"
```

Make it executable:

```
chmod +x create_instances.sh
```

E2. `delete_instances.sh`

```
nano delete_instances.sh
```

Paste in:

```
#!/bin/bash
set -e

export $(grep -v '^#' .env | xargs)

if [ ! -f instance_ids.txt ]; then
    echo "No instance_ids.txt found. Nothing to delete."
    exit 1
fi

IDS=$(cat instance_ids.txt | tr '\n' ' ')

echo "Terminating instances: $IDS"

aws ec2 terminate-instances \
    --instance-ids $IDS \
    --region "$AWS_REGION"

echo "Waiting for instances to terminate..."
aws ec2 wait instance-terminated \
    --instance-ids $IDS \
    --region "$AWS_REGION"

echo "All instances terminated successfully."
rm instance_ids.txt
```

Make it executable:

```
chmod +x delete_instances.sh
```

E3. Run the Full Cycle

```
./create_instances.sh
```

Confirm both instances are running:

```
aws ec2 describe-instances \
    --filters "Name=tag:Name,Values=$NAME_TAG" "Name=instance-state-name,Values=running" \
    --query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress]" \
    --output table \
    --region us-east-2
```

Then clean up:

```
./delete_instances.sh
```

✓ **Checkpoint:** `create_instances.sh` reports 2 instance IDs; `delete_instances.sh` waits and confirms termination; `instance_ids.txt` is removed afterward.

E4. Commit Your Scripts (Not Your `.env` !)

```
git add create_instances.sh delete_instances.sh
git status # Double-check .env is NOT listed here
git commit -m "Add create and delete scripts for EC2 instances using env variables"
git push
```

8. Troubleshooting

Issue	Likely Cause	Fix
<code>InvalidAMIID.NotFound</code>	AMI ID copied incorrectly, or from wrong region	Re-run the <code>describe-images</code> command in Part B1 for your exact region
<code>UnauthorizedOperation</code>	IAM user lacks EC2 permissions	Ask instructor to confirm your IAM policy includes EC2 actions
Instance stuck in <code>pending</code> for a long time	Normal — can take 1-2 minutes	Wait and re-check with <code>describe-instances</code>
Security group rule fails: “already exists”	Rule was already added in a previous run	Skip that command or check existing rules with <code>describe-security-groups</code>
<code>\$AMI_ID</code> prints empty/nothing	<code>.env</code> file not loaded in current shell session	Re-run <code>export \$(grep -v '^#' .env xargs)</code>
<code>.env</code> shows up in <code>git status</code> anyway	<code>.gitignore</code> created after <code>.env</code> was already tracked once	Run <code>git rm --cached .env</code> then commit again
Delete script says “No instance_ids.txt found”	Create script wasn’t run first, or file was deleted	Re-run <code>create_instances.sh</code> , or manually terminate via <code>describe-instances + terminate-instances</code>
Charges appear on AWS billing dashboard	Instances left running past Free Tier hours	Always run <code>delete_instances.sh</code> ; set a Billing Alarm (see Lab 01, Section 6)

9. Submission Checklist

Submit a short lab report including:

- ☐ Screenshot of a decomposed `aws ec2 describe-instances` command with each part labeled (service/operation/parameters).
- ☐ Screenshot of the “Hello from...” web page from Example 2 (Part C5).
- ☐ Screenshot of `git status` showing `.env` is **not** tracked, alongside your `.gitignore` file contents.
- ☐ Screenshot of `create_instances.sh` output showing 2 instance IDs.
- ☐ Screenshot of `delete_instances.sh` output confirming termination.
- ☐ Link to your GitHub repo showing `create_instances.sh`, `delete_instances.sh`, `.env.example`, and `.gitignore` (but **not** `.env`).
- ☐ One paragraph explaining why environment files are excluded from Git, even in a “private” repository.

10. Key Terms Glossary

- **AMI (Amazon Machine Image)** – A template containing the OS and pre-installed software used to launch an EC2 instance.
- **Security Group** – A virtual firewall controlling inbound/outbound traffic to an instance.
- **User-data** – A script passed to an instance at launch time that runs automatically on first boot.
- **Tag** – A key-value label attached to AWS resources for organization, filtering, and billing.
- **Environment file (`.env`)** – A file storing configuration values as `KEY=value` pairs, loaded into a script’s environment at runtime.
- **`.gitignore`** – A file listing paths Git should never track or commit, commonly used to exclude secrets and environment-specific files.
- **JMESPath (`--query`)** – A query language used by the AWS CLI’s `--query` flag to extract specific fields from JSON output.